

Lecture 1: Overview of Programming Languages.

1. Phân loại Ngôn ngữ: Imperative vs. Declarative (Tư duy Lập trình)

Đây là kiến thức quan trọng nhất để phân biệt "thợ code" và "kỹ sư".

- **Imperative (Mệnh lệnh) - "How to do":**
 - **Định nghĩa:** Em ra lệnh cho máy tính làm từng bước một để đạt kết quả.
 - **Đại diện:** C, Java (cách viết cũ), Pascal.
+3
 - **Ví dụ:** Để tính $3 * x$, em viết $x + x + x$.
 - **Đặc điểm:** Dựa trên kiến trúc Von Neumann (thay đổi trạng thái biến trong bộ nhớ).
+1
- **Declarative (Khai báo) - "What to do":**
 - **Định nghĩa:** Em chỉ mô tả kết quả mong muốn, không quan tâm máy tính làm thế nào.
+1
 - **Đại diện:** SQL, HTML, các ngôn ngữ Functional (Haskell).
+1
 - **Ví dụ:** `SELECT * FROM table` (Em không cần viết vòng lặp để tìm dữ liệu).

💡 **Góc nhìn thực tế:** Java hiện đại đang chuyển dịch mạnh sang **Declarative** (thông qua Stream API, Annotation trong Spring Boot). Hiểu sự khác biệt này giúp em viết code gọn và "sang" hơn.

2. Các Mô hình Lập trình (Programming Paradigms)

Đừng hiểu lầm cái này thay thế cái kia. Một Backend Dev giỏi phải biết dùng cả 4 món này:

1. **Procedural (Thủ tục):** Code chạy tuần tự. Nền tảng cơ bản nhất.
2. **Object-Oriented (Hướng đối tượng - OOP):** Tổ chức code thành các Object tương tác với nhau. Đây là trọng tâm của Java và môn học này.
+1
3. **Functional (Hàm):** Tập trung vào hàm thuần khiết (pure functions), không thay đổi dữ liệu (immutability). Giúp xử lý song song tốt hơn.
+1
4. **Concurrent (Đồng thời):** Xử lý nhiều tác vụ cùng lúc. Rất quan trọng cho các hệ thống lớn cần hiệu năng cao.
+1

💡 **Lưu ý:** Các ngôn ngữ hiện đại như Java, Python là **Multi-paradigm** (Đa mô hình). Chúng hỗ trợ cả OOP, Functional và Concurrent cùng lúc.

3. Cách Ngôn ngữ Thực thi (Implementation)

Hiểu cái này để biết tại sao Java lại "viết một lần, chạy mọi nơi".

- **Compiler (Biên dịch):** Dịch toàn bộ source code sang mã máy (Machine code) rồi mới chạy. Chạy nhanh nhưng code phụ thuộc vào phần cứng cụ thể (Ví dụ: C, C++).
+1
 - **Interpreter (Thông dịch):** Đọc dòng nào chạy dòng đó. Chạy chậm hơn nhưng linh hoạt (Ví dụ: Python, Ruby).
 - **Hybrid (Lai): Đây là Java.**
 - Dùng Compiler để dịch ra **Bytecode** (file `.class`).
 - Dùng JVM (máy ảo) để chạy Bytecode đó.
 - **Lợi ích:** Cân bằng giữa hiệu năng và tính di động (Portability).
+2
-

Tóm tắt cho bài thi (Exam Focus)

Nếu gặp câu hỏi trắc nghiệm, hãy nhớ các từ khóa này:

- **Von Neumann Architecture:** Gồm CPU (xử lý) + Memory (chứa lệnh và dữ liệu). Đây là cơ sở của các ngôn ngữ Imperative.
- **Trade-offs (Đánh đổi):** Không có ngôn ngữ nào hoàn hảo. Luôn phải đánh đổi giữa *Hiệu năng (Performance)* và *An toàn (Safety)*; *Dễ học* và *Sức mạnh diễn đạt*.
+1
- **Java thuộc nhóm nào?** Java là ngôn ngữ **Imperative, Object-Oriented, Hybrid implementation** (biên dịch ra bytecode), và mạnh về **Portability**.
+2

1. Programming Language (PL) là gì

- PL = công cụ để **diễn đạt thuật toán** cho máy
- Không chỉ là cú pháp, mà là **cách con người suy nghĩ khi giải quyết vấn đề**

👉 Đi thi: đừng trả lời “ngôn ngữ để lập trình” (rất non).

2. Vì sao có nhiều ngôn ngữ lập trình

3 lý do chính (nhớ đúng 3 ý này):

1. **Evolution** – sửa nhược điểm của ngôn ngữ cũ
2. **Purpose** – phục vụ bài toán khác nhau
3. **Preference** – gu & cộng đồng

👉 Không có “ngôn ngữ tốt nhất”, chỉ có **phù hợp**.

3. Programming Paradigm (cực kỳ quan trọng)

Paradigm = cách tư duy khi viết chương trình

Các paradigm chính (theo thứ tự tiến hoá):

1. Procedural – tư duy theo bước
2. Object-Oriented – tư duy theo đối tượng
3. Functional – tư duy biến đổi dữ liệu
4. Concurrent – tư duy song song
5. Logic-based – tư duy theo luật

👉 Paradigm mới **không xoá paradigm cũ**, mà mở rộng cách nghĩ.

4. Declarative vs Imperative (hay ra thi)

- **Imperative**: nói *làm thế nào*
→ Java, C, Python
- **Declarative**: nói *muốn cái gì*
→ SQL, HTML

👉 Java là **imperative**, OOP giúp code **giống declarative** hơn.

5. Von Neumann Programming Languages

Von Neumann = CPU đọc lệnh tuần tự và thay đổi dữ liệu trong bộ nhớ.

Ý chính phải nhớ:

- Dựa trên kiến trúc:
 - Memory (RAM)
 - CPU
- Chương trình = **thay đổi giá trị biến trong bộ nhớ**

Ví dụ ngôn ngữ:

- Past: Fortran, C
- Present: Java, Python, JavaScript

👉 OOP **vẫn là Von Neumann**, chỉ tổ chức code tốt hơn.

6. Subtypes của Von Neumann PLs

- **Scripting PLs**: JS, Python
→ viết nhanh, trừu tượng cao
 - **OOP PLs**: Java, C#
→ quản lý hệ thống lớn
-

7. Language Design Trade-offs (câu ăn đũa)

Không có ngôn ngữ hoàn hảo. Chỉ có **đánh đổi**:

- Performance vs Safety → C vs Java
- Ease of learning vs Expressive power → Python vs Haskell
- Portability vs Optimization → Java vs C++

👉 Trả lời kiểu kỹ sư = **luôn nói trade-off + bối cảnh**.

8. Compiler vs Interpreter vs Hybrid

- Compiler: C, C++
- Interpreter: Python
- Hybrid: Java (bytecode + JVM)

👉 Ảnh hưởng đến performance, portability, debugging.

9. Vì sao phải học nguyên lý ngôn ngữ lập trình

- Học ngôn ngữ mới nhanh hơn
 - Chọn ngôn ngữ đúng cho bài toán
 - Hiểu vì sao ngôn ngữ được thiết kế như vậy
-

10. Liên hệ trực tiếp tới PR2 (rất quan trọng)

- PR2 không kiểm tra Java syntax
- PR2 kiểm tra:
 - Bạn đang dùng **paradigm** nào
 - Bạn có tư duy **OOP** đúng hay không

👉 Nhét hết vào `main()` = procedural ❌

👉 Chia class đúng trách nhiệm = OOP ✅

11. 5 câu chốt NÊN HỌC THUỘC

1. Paradigm là cách tư duy khi lập trình.
2. Không có ngôn ngữ tốt nhất, chỉ có phù hợp.

3. Java là imperative, Von Neumann language.
4. OOP không thay thế imperative, nó tổ chức nó.
5. Mọi ngôn ngữ đều là kết quả của trade-offs.